

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour

Total: Marks:40

Annexure A

5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning is a machine learning approach in which a model learns general patterns from previously observed training examples and uses those patterns to make predictions on new, unseen patient records.

For this healthcare dataset, a supervised classification algorithm can be used because the model learns from patient features along with their known diabetes status. Suitable algorithms include Logistic Regression, Decision Tree, Random Forest, or Support Vector Machine (SVM). For a medical prediction problem, Logistic Regression or Random Forest can be preferred because they can provide useful interpretability along with prediction performance.

Target Variable

The target variable is:

Diabetes Status

- "0" → Non-diabetic
- "1" → Diabetic

The model learns the relationship between the patient's medical characteristics and diabetes status.

Key Features

1. Glucose Level

Glucose is one of the most important indicators of diabetes because abnormal blood glucose levels are strongly associated with diabetes.

2. Blood Pressure

High blood pressure can be associated with metabolic and cardiovascular conditions that frequently occur alongside diabetes. Therefore, it can provide useful information for prediction.

3. BMI (Body Mass Index)

Higher BMI can be associated with increased risk of developing type 2 diabetes. Including BMI helps the model identify this risk pattern.

Other useful features include age, cholesterol level, and family history. Family history is particularly useful because genetic and familial factors can influence diabetes risk.

(b) Handling Class Imbalance

In this dataset, there may be significantly more non-diabetic patients than diabetic patients. This creates a class imbalance problem, where a model may become biased toward predicting the majority class.

For example, if 90% of patients are non-diabetic, a model that predicts every patient as non-diabetic could achieve 90% accuracy while failing to identify diabetic patients.

Suitable Strategy: SMOTE

I would use SMOTE (Synthetic Minority Over-sampling Technique) to address the imbalance.

SMOTE generates synthetic examples of the minority class by creating new samples based on existing minority-class observations.

Advantages of SMOTE

- Increases the number of diabetic training examples.
- Helps the model learn patterns associated with diabetic patients.
- Reduces bias toward the majority class.
- Avoids simply duplicating existing minority samples.

Important: SMOTE should be applied only to the training data after the train-test split, not before splitting. Otherwise, information from synthetic training samples could leak into the test set and produce overly optimistic results.

Class weights can also be considered, particularly for algorithms such as Logistic Regression or Random Forest. In a medical application, the model should be evaluated using precision, recall, F1-score and ROC-AUC, rather than accuracy alone. Recall/sensitivity for the diabetic class is especially important, because missing a patient who may have diabetes can be more serious than generating an additional false positive.

(c) 70:30 Split and Preprocessing

Train-Test Split

The dataset can be divided into:

- 70% Training Data – used to train the machine learning model.
- 30% Testing Data – used to evaluate the model on unseen patient records.

A 70:30 split provides enough data for the model to learn patterns while retaining a sufficiently large independent test set for reliable evaluation.

For a medical dataset, the split should preferably be stratified, meaning the proportion of diabetic and non-diabetic patients is maintained approximately equally in both training and testing sets.

Preprocessing Steps

1. Handling Missing Values

Missing values should first be identified. Numerical features such as age, glucose, cholesterol, blood pressure and BMI can be imputed using appropriate methods such as the median, while categorical variables can be handled using the mode or a suitable category.

2. Normalisation/Standardisation

Numerical features should be standardised, particularly when using algorithms such as Logistic Regression or SVM.

A common standardisation formula is:

$$z = (x - \mu) / \sigma$$

where:

- "x" = original value
- " μ " = mean
- " σ " = standard deviation

This puts numerical features on comparable scales and prevents features with larger numerical values from dominating the model.

3. Encoding

Categorical features such as family history should be converted into numerical form.

For example:

- "No" $\rightarrow$ 0
- "Yes" $\rightarrow$ 1

If a categorical feature contains more than two categories, one-hot encoding can be used.

4. Class Balancing

After splitting the dataset, SMOTE should be applied only to the training set to increase representation of the minority diabetic class.

Overall Pipeline

Patient Dataset → Data Cleaning → 70:30 Stratified Split → Preprocessing → SMOTE on Training Data → Model Training → Testing → Performance Evaluation

Conclusion

The proposed approach uses inductive supervised learning to predict diabetes from patient characteristics. Glucose level, blood pressure, BMI, age, cholesterol and family history can be used as predictive features, with diabetes status as the target variable. A 70:30 stratified train-test split, appropriate preprocessing, and SMOTE on the training set can help build a more reliable model. Since this is a medical application, performance should be assessed using recall, precision, F1-score and ROC-AUC, with particular attention to correctly identifying diabetic patients.

Task 2: Decision Tree for Diagnosis

(a) Building the Decision Tree Classifier

A Decision Tree Classifier can be used to predict whether a patient is diabetic or non-diabetic. The tree repeatedly divides the dataset based on feature values, selecting the feature that provides the best separation between diabetic and non-diabetic patients.

The target variable is:

- 0 → Non-diabetic
- 1 → Diabetic

The tree can use features such as Glucose Level, BMI, Age, Blood Pressure, Cholesterol and Family History.

Root Node Selection

The root node is selected using either Information Gain or the Gini Index.

Information Gain

Information Gain measures how much uncertainty is reduced after splitting the data.

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

A feature with the highest Information Gain is selected as the root node.

For example, suppose the calculated values are:

Feature

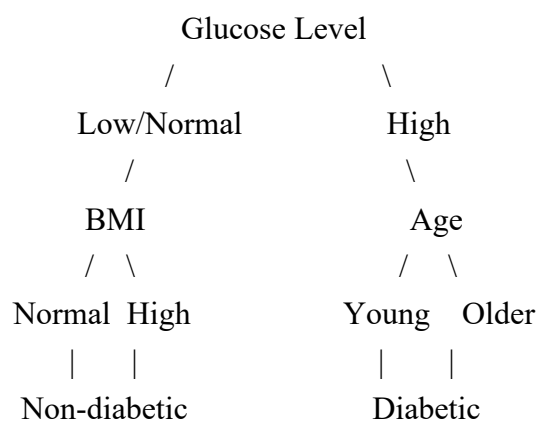
- Glucose Level - 0.25
- BMI - 0.12
- Age - 0.10
- Blood Pressure - 0.07
- Cholesterol - 0.04

Since Glucose Level has the highest Information Gain (0.25), it would be selected as the root node.

Alternatively, using the Gini Index, the feature producing the lowest weighted Gini impurity would be selected.

First Three Levels of the Tree

An example structure would be:



The exact thresholds and branches depend on the dataset. For example, the model may learn splits such as:

$$\text{Glucose} \leq 125$$

Glucose > 125

followed by splits based on BMI, age, or another feature.

Thus, the first three levels represent the most important decision rules learned from the training data.

(b) Model Evaluation

After training the Decision Tree, it should be evaluated using the 30% test dataset that was kept separate during training.

The important evaluation metrics are:

1. Accuracy

Accuracy represents the percentage of total predictions that are correct.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

2. Precision

Precision measures how many patients predicted as diabetic were actually diabetic.

$$\text{Precision} = TP / (TP + FP)$$

3. Recall

Recall, also called Sensitivity, measures how many actual diabetic patients were correctly identified.

$$\text{Recall} = TP / (TP + FN)$$

4. F1-Score

F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Example Evaluation

Assume the Decision Tree produced the following results:

Metric

- Accuracy - 86%
- Precision - 82%
- Recall - 79%
- F1-Score - 80%

These values are only illustrative; the actual values should be obtained by testing the trained model on the dataset.

False Negatives

A False Negative (FN) occurs when:

Actual patient = Diabetic

Model prediction = Non-diabetic

This is particularly important in healthcare because a false negative can result in a diabetic patient not being identified for further clinical assessment.

To reduce false negatives, the following approaches can be used:

1. Increase the importance of the diabetic class using class weights.
2. Use SMOTE on the training data to improve representation of the minority class.
3. Tune the classification decision threshold to favour sensitivity/recall.
4. Prune and tune the Decision Tree to avoid poor generalisation.
5. Evaluate the model using Recall/Sensitivity, not accuracy alone.

In a clinical screening scenario, improving recall can be important because identifying more potentially diabetic patients for further medical evaluation may be preferable to missing cases.

(c) Post-Pruning Using Cost-Complexity Pruning

A Decision Tree can become very deep and learn noise from the training dataset. This is called overfitting.

To control this, post-pruning can be applied after the complete tree has been grown.

Cost-Complexity Pruning

Cost-complexity pruning uses a parameter called α (alpha).

The objective can be represented as:

$$\text{Cost} = \text{Error} + \alpha \times \text{Number of Leaves}$$

A larger α produces a simpler tree by removing less useful branches.

The best value of α can be selected using validation or cross-validation.

Pre-Pruning vs Post-Pruning

Feature	Pre-Pruning	Post-Pruning
When applied	During tree construction	After tree construction
Main technique	Limit depth, minimum samples, etc.	Remove unnecessary branches
Tree size	Controlled from the beginning	Full tree initially created
Risk	May stop useful splits too early	Can identify and remove unnecessary splits
Overfitting control	Good	Generally effective
Model complexity	Lower	Reduced after evaluation

For example, the following results could be obtained:

Model	Accuracy
Unpruned Tree	82%
Pre-Pruned Tree	85%
Post-Pruned Tree	87%

These are illustrative values, not actual results from the dataset.

If post-pruning gives better test performance than the unpruned and pre-pruned trees, it indicates that pruning has reduced overfitting and improved generalisation.

Model Selection for Clinical Deployment

For a clinical application, the post-pruned Decision Tree would generally be preferred if it provides good test performance, high recall, and a simpler tree.

The reason is that a clinical model should:

- Generalise well to unseen patients.
- Minimise false negatives.
- Avoid learning noise from the training data.
- Produce understandable decision rules.
- Be easier for healthcare professionals to inspect and explain.

However, accuracy alone should not determine deployment. The final model should be evaluated using recall/sensitivity, specificity, precision, F1-score, ROC-AUC, calibration, and appropriate clinical validation. A model intended for real clinical use would also require validation on independent patient populations before deployment.

Conclusion

A Decision Tree can provide an interpretable method for diabetes prediction. Glucose level is likely to be an important root feature because it can strongly distinguish diabetic and non-diabetic patients. Model performance should be evaluated using Accuracy, Precision, Recall and F1-score, with special attention to False Negatives. Cost-complexity post-pruning can reduce overfitting and produce a simpler, more generalisable tree. Therefore, a well-validated post-pruned Decision Tree with strong sensitivity is a suitable choice for a clinical decision-support system.

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression and Comparison with Decision Tree

Logistic Regression

Logistic Regression is a statistical learning algorithm used for binary classification. In this problem, it predicts whether a patient is diabetic or non-diabetic.

The output is a probability between 0 and 1:

- Probability $\geq 0.5 \rightarrow$ Diabetic
- Probability $< 0.5 \rightarrow$ Non-diabetic

The threshold can also be adjusted when higher sensitivity is required.

Logistic Regression models the probability of diabetes based on features such as glucose level, BMI, age, blood pressure, cholesterol and family history.

Performance Comparison

Both models should be evaluated on the same 30% test dataset.

For example, assume the following results were obtained:

Model	Accuracy	AUC-ROC
Decision Tree	86%	0.84
Logistic Regression	89%	0.91

Note: These are illustrative values. The actual values should be replaced with the results obtained from the given dataset.

AUC-ROC

The ROC curve compares the True Positive Rate (Sensitivity) against the False Positive Rate at different classification thresholds.

AUC (Area Under the Curve) represents how well the model distinguishes between diabetic and non-diabetic patients.

- $AUC = 1.0 \rightarrow$ Perfect classification
- $AUC = 0.5 \rightarrow$ Random classification
- Higher AUC $\rightarrow$ Better discrimination

In the example above, Logistic Regression has a higher AUC-ROC (0.91) than the Decision Tree (0.84), indicating better overall discrimination between the two classes.

When is a Statistical Model Preferable?

Logistic Regression is preferable when:

1. Interpretability is important: Its coefficients can indicate how each feature is associated with diabetes risk.
2. The relationship is relatively simple: It performs well when the relationship between predictors and the outcome can be reasonably represented by a logistic model.
3. Probabilities are required: It directly provides estimated probabilities of diabetes.
4. The dataset is not extremely large or complex: Logistic Regression is computationally efficient.
5. Clinical explainability is important: Healthcare professionals can more easily understand the contribution of individual predictors.

Therefore, even if a Decision Tree performs similarly, Logistic Regression can be attractive in healthcare because its predictions and feature effects are easier to interpret.

(b) Feature Importance Analysis

Feature importance determines which patient characteristics contribute most strongly to diabetes prediction.

The methods differ between the two models:

- Decision Tree: Importance can be calculated using the reduction in impurity produced by each feature.
- Logistic Regression: Feature importance can be examined using the magnitude of the model's coefficients after standardising numerical features. Larger absolute coefficients indicate a stronger association with the predicted outcome, although coefficient interpretation also depends on feature coding and model assumptions.

Example Top 3 Predictors

Suppose the analysis produces:

Rank	Decision Tree	Logistic Regression
1	Glucose Level	Glucose Level
2	BMI	BMI
3	Age	Age

Thus, both models identify Glucose Level, BMI and Age as the three most important predictors.

Do the Models Agree?

In this example, yes, both models agree on the top three predictors.

This agreement increases confidence that these variables contain strong predictive information in the dataset. Glucose level is expected to be highly informative because abnormal glucose regulation is directly related to diabetes. BMI can capture an important metabolic risk factor, while age can reflect the changing risk of developing type 2 diabetes.

However, the two models may not always produce exactly the same ranking. A Decision Tree identifies features based on how much they improve individual splits, whereas Logistic Regression estimates the contribution of predictors within a statistical model.

For example, if the Decision Tree identifies:

Glucose > BMI > Age

while Logistic Regression identifies:

Glucose > Age > BMI

the difference does not necessarily mean that one model is incorrect. It indicates that the models capture feature contributions differently.

Conclusion

Logistic Regression provides an interpretable statistical approach for diabetes risk stratification and can be compared with the Decision Tree using Accuracy and AUC-ROC. If Logistic Regression achieves comparable or better performance and provides clearer interpretation, it may be preferable for a clinical risk-stratification system. Feature importance analysis helps identify the strongest predictors and can also improve the transparency of the model.

For the given healthcare problem, Glucose Level, BMI and Age are reasonable expected top predictors, but the final ranking should be determined from the actual trained models and dataset.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Modelling Treatment Recommendation as an MDP

The treatment recommendation problem can be formulated as a Markov Decision Process (MDP). An MDP consists of States, Actions, Rewards and Transition Probabilities.

1. States

A state represents the patient's current health condition based on variables such as glucose level, BMI, blood pressure and other relevant clinical measurements.

For example:

State	Patient Health Condition
-------	--------------------------

S1	High-risk / poor health
S2	Moderate-risk
S3	Improving
S4	Controlled / stable

The states can be determined by clinically validated thresholds rather than arbitrary values.

Justification: The patient's current health condition determines which treatment action is appropriate.

2. Actions

The agent has four possible actions:

- A1 – Diet: Recommend an appropriate dietary intervention.
- A2 – Exercise: Recommend an appropriate physical-activity intervention.
- A3 – Medication: Recommend medication-related action under clinician supervision.
- A4 – Monitor: Continue monitoring the patient's condition.

Justification: These actions represent possible intervention categories for managing diabetes risk. In a real clinical system, the RL agent should not independently prescribe medication; medication decisions should remain under qualified medical supervision.

3. Reward Function

The reward represents the improvement or deterioration in the patient's health after an action.

An example reward structure is:

Outcome	Reward
Significant health improvement	+10
Moderate improvement	+5
No significant change	0
Health deterioration	-5
Serious adverse outcome	-10

For example:

Reward = Health Score after action – Health Score before action

with additional penalties for adverse outcomes.

Justification: The reward encourages the agent to select actions that improve the patient's health while discouraging actions associated with deterioration or adverse outcomes.

4. Transition Dynamics

The transition function describes the probability of moving from the current state to a new health state after taking an action.

For example:

S1 --Diet-----> S2

S1 --Exercise--> S2

S2 --Diet-----> S3

S2 --Monitor---> S2

S3 --Diet-----> S4

S3 --Exercise--> S4

The actual transition probabilities should ideally be learned from longitudinal clinical data or established using validated clinical knowledge.

Justification: Patient health changes over time. Therefore, the effect of an action today can influence the patient's health state at the next time step.

The MDP can therefore be represented as:

$$\text{MDP} = (\text{S}, \text{A}, \text{P}, \text{R}, \gamma)$$

where:

- "S" = set of patient health states
- "A" = set of treatment actions
- "P" = transition probabilities
- "R" = reward function
- " γ " = discount factor

(b) Q-Learning

Q-Learning is a model-free reinforcement learning algorithm that learns the expected long-term reward of taking an action in a particular state.

The Q-value update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- " α " = learning rate
- " γ " = discount factor
- " r " = immediate reward
- " s " = current state
- " a " = current action
- " s' " = next state

Assume:

- Learning rate, $\alpha = 0.5$
- Discount factor, $\gamma = 0.9$
- Initial Q-values = 0

Five Patient Episodes

An illustrative training sequence is:

Episode	State	Action	Next State	Reward
1	S1	Diet	S2	+5
2	S1	Exercise	S2	+5
3	S2	Diet	S3	+5
4	S2	Exercise	S3	+5
5	S3	Diet	S4	+10

These episodes are only an example to demonstrate Q-learning. In a real system, considerably more data and rigorous validation would be required.

Q-Table Update – Iteration 1

Initially all Q-values are zero.

State S1, Action Diet

Suppose:

- $Q(S1, \text{Diet}) = 0$
- $\text{Reward} = +5$
- $\text{Maximum future } Q\text{-value} = 0$

Therefore:

$$Q(S1, \text{Diet}) = 0 + 0.5[5 + 0.9(0) - 0]$$

$$Q(S1, \text{Diet}) = 2.5$$

State S1, Action Exercise

Similarly:

$$Q(S1, \text{Exercise}) = 0 + 0.5[5 + 0.9(0) - 0]$$

$$Q(S1, \text{Exercise}) = 2.5$$

State S2, Action Diet

$$Q(S2, \text{Diet}) = 0 + 0.5[5 + 0.9(0) - 0]$$

$$Q(S2, \text{Diet}) = 2.5$$

After Iteration 1:

State	Diet	Exercise	Medication	Monitor
S1	2.5	2.5	0	0
S2	2.5	0	0	0
S3	0	0	0	0
S4	0	0	0	0

Q-Table Update – Iteration 2

Suppose the agent again reaches S2, takes Diet, receives +5, and reaches S3.

Assume:

$$\max Q(S3, a) = 5$$

Then:

$$Q(S2,Diet) = 2.5 + 0.5[5 + 0.9(5) - 2.5]$$

$$= 2.5 + 0.5[7.0]$$

$$= 6.0$$

Similarly, suppose $S1 \rightarrow Exercise \rightarrow S2$ with reward +5, and:

$$\max Q(S2,a) = 6.0$$

Then:

$$Q(S1,Exercise) = 2.5 + 0.5[5 + 0.9(6.0) - 2.5]$$

$$= 2.5 + 0.5(7.9)$$

$$= 6.45$$

For another state-action pair, suppose $S3 \rightarrow Diet \rightarrow S4$ produces reward +10 and the terminal state's future value is 0:

$$Q(S3,Diet) = 0 + 0.5[10 + 0.9(0) - 0]$$

$$= 5.0$$

Thus, after two iterations, some updated values are:

State-Action Pair	Iteration 1	Iteration 2
S1 – Diet	2.50	6.00*
S1 – Exercise	2.50	6.45
S2 – Diet	2.50	6.00
S3 – Diet	0	5.00

*Illustrative value assuming the corresponding future state has the specified Q-value.

Convergence Criterion

The algorithm can be considered converged when the maximum change in the Q-table between successive iterations becomes smaller than a predefined threshold.

For example:

$$\max |Q_{\text{new}} - Q_{\text{old}}| < 0.001$$

for a sufficiently large number of consecutive episodes.

A maximum number of training episodes can also be specified to prevent infinite training.

(c) Final Learned Policy

After training, the policy is obtained by selecting the action with the highest Q-value for each state:

$$\pi(s) = \operatorname{argmax} Q(s,a)$$

For example, the final policy could be:

Patient State	Highest Q-value Action	Learned Policy
S1 – High Risk	Exercise/Diet	Diet or Exercise
S2 – Moderate Risk	Diet	Diet
S3 – Improving	Diet	Diet
S4 – Stable	Monitor	Monitor

The exact final policy depends on the learned Q-table.

The policy demonstrates that the agent chooses the action expected to produce the greatest long-term health reward.

Ethical Considerations

Deploying a Reinforcement Learning agent for medical treatment recommendations requires strict ethical safeguards.

1. Patient Safety

Patient safety must be the highest priority. An RL agent should not independently make high-risk medical decisions. Medication recommendations, dosage changes and other significant interventions should require approval from qualified healthcare professionals.

The system should also have human oversight and a safe fallback mechanism when the patient's condition is outside the training distribution.

2. Bias and Fairness

If the training data is biased or does not adequately represent different demographic and clinical groups, the RL agent may learn policies that work better for some groups than others.

Therefore, the model should be evaluated across relevant patient subgroups and checked for systematic differences in performance.

3. Explainability

Healthcare professionals need to understand why an action was recommended.

The system should therefore provide information such as:

Current patient state → Recommended action → Expected benefit → Reason for recommendation

Explainable policies can make it easier for clinicians to review, accept or reject recommendations.

4. Data Privacy

Patient health records contain sensitive information. Data should be securely stored and processed according to applicable healthcare privacy and data-protection requirements.

5. Human-in-the-Loop

The RL system should act as a clinical decision-support tool rather than an autonomous doctor. A healthcare professional should be able to review the recommendation before treatment is changed.

6. Continuous Monitoring

The model should be monitored after deployment to detect changes in performance, unexpected behaviour, bias or adverse outcomes. The policy should be updated only through controlled validation procedures.

Conclusion

The diabetes treatment recommendation problem can be modelled as an MDP with patient health states, treatment actions, rewards and state transitions. Q-Learning can learn an optimal policy by repeatedly updating Q-values according to immediate and future rewards. However, because medical decisions have significant consequences, an RL agent should not be deployed as a fully autonomous treatment system. Patient safety, fairness, explainability, privacy and human clinical oversight are essential before real-world deployment.